ONSITE STORAGE · OSS-NEXT · AGENTIC-AI READINESS

Making the storefront legible to AI agents

Final report — 50 of 54 tasks complete, 93%. All nine phases built.

Date 13 Aug 2026 **Branch** integrate-wp-render **Tracker** `docs/audits/AGENTIC_READINESS.md` **Status** uncommitted working tree

The goal is that AI search crawlers, assistants and shopping agents can find, read and correctly interpret this site. The hard part is structural: ~1,700 of the ~12,000 URLs are WordPress pages rendered as injected Elementor markup, so nothing could be fixed page by page — the fixes had to go into the conversion pipeline and the catch-all route.

50/54	**0**	**4**	**12**	**45**
TASKS COMPLETE	REMAINING, BUILDABLE	BLOCKED ON OTHERS	DEFECTS FOUND	AUTOMATED CHECKS

Progress by phase

PHASE	DONE	STATE
0 · Decisions	4/4	COMPLETE All four answered and recorded
1 · Access & discovery	6/6	COMPLETE
2 · Machine-readable content	6/7	1 BLOCKED needs a Django field
3 · Structured data	8/9	1 BLOCKED needs return-policy terms
4 · Agent API	6/6	COMPLETE 3 endpoints + OpenAPI + rate limiting
5 · MCP server	5/5	COMPLETE stateless Streamable HTTP, 4 tools
6 · Agentic commerce	5/5	COMPLETE quote requests; no agent checkout, per D3
7 · Page hygiene	5/6	1 BLOCKED WordPress content work
8 · Measurement & CI	5/6	1 BLOCKED manual, needs live site
Total	**50/54**	**93% — every buildable task done**

What is now live

Agents can be admitted, find the site, and read it

- **Crawler policy** — `robots.txt` now names 17 agents individually, each with a recorded reason. Search and user-initiated agents allowed; training-only crawlers blocked.
- **/llms.txt and /llms-full.txt** — a curated index (123 links, all verified) and a full-text variant. Curated, not a sitemap dump: hand-ranked key pages, the depot network, and a stride-sampled set of guides.
- **Markdown view of every page** — `/any-page.md` or `Accept: text/markdown`. The same content at roughly a twentieth of the bytes, with no guesswork about which `<div>` was a heading.

Structured data is complete and interconnected

- Every page emits one `@graph` with a canonical Organization and WebSite at a stable `@id`, referenced rather than restated — so "the seller of this container" resolves to the company described on the homepage.
- Added: BreadcrumbList on PDP and all WordPress pages, FAQPage on the PDP, ItemList on the listing page, LocalBusiness with real service areas on 55 depot pages.
- **Recovered the structured data on ~1,700 WordPress pages** that the conversion pipeline had been silently destroying (see F1).

There is an API written for models, not for our own front end

- **Three read endpoints** under `/api/agent/v1/` — search, product detail, and delivery availability by zip. Flat JSON, no envelope, versioned from the first request. The existing search API speaks InstantSearch's transport shape, which no agent would decode.
- **Every price is an object**, never a bare number: `amount`, `basis`, a plain-English `description`, and `asOf` / `validUntil` timestamps. A model that quotes the description cannot repeat F3.
- **`/openapi.json`** — hand-written 3.1, with the descriptions written as instructions to a model rather than as reference text.
- **Rate limited** on the existing Upstash instance — 60/min reads, 5/min quotes, fails open. Verified under 75 parallel requests: exactly 60 served, 15 refused with `Retry-After`.

Claude and ChatGPT can connect to the store directly

- **`/api/mcp`** — a stateless Model Context Protocol server over Streamable HTTP, negotiating three protocol revisions. No key, no OAuth: paste the URL in as a custom connector.
- **Four tools** mapping 1:1 onto the API above, so there is one behaviour to maintain, not two. Each returns prose a model can quote as well as the structured payload.
- Tool calls are logged by name, so we can see what agents actually ask for.

An agent can start a sale — and cannot finish one

- **`POST /api/agent/v1/quote`** — an agent submits a quote request for a customer, tagged agent-originated, answered with the nearest depot. Per decision D3 there is no ordering or payment endpoint, and the policy page says so plainly.
- **`/api/feeds/products.jsonl`** — the whole catalog, one product per line, paginated. Verified at 10,192 products with a price basis on every single one.
- **The Google Merchant feed no longer misreports rentals** — the description leads with the fact that the figure is monthly, states the term, and gives the total over the term; `custom_label_0/1` carry it as data. 6,720 rent-to-own, 1,260 rental, 2,212 purchase.
- **`/agents`** — one page stating what agents may do, the rate limits, the rental-price trap, and how to ask for more. Generated from the same config the code enforces, so it cannot drift.

It is measurable, and regressions get caught

- **Agent traffic logging** into Redis from the proxy, with an admin report at `/admin/agent-traffic`. Costs human traffic nothing.
- **`npm run audit:agentic`** — 45 checks covering schema validity, heading and landmark structure, both Markdown mechanisms, real 404s, robots.txt contents, every machine-readable route and all 123 llms.txt links. Exits non-zero, so it can gate CI.

Defects found along the way

None of these were on the original task list. They were found by building the checks and then actually running them.

#	DEFECT	RESOLUTION
F1	All ~1,700 WordPress pages shipped **zero structured data** — a `$('script').remove()` added to fix a reCAPTCHA crash was also deleting every `ld+json` block	FIXED Schema extracted before the strip. 3 of 11 sampled pages recovered an FAQPage
F2	Missing WordPress paths returned **HTTP 200** with the Not Found page — invisible to any status-code-based link check	FIXED Real 404s via a fail-open proxy check. 39/39 real pages verified unaffected
F3	The **monthly rental price** problem: ~8,000 of ~10,000 products carry a monthly figure in `sale_price`. A 40ft container read as "$232.14"	FIXED One `getPriceBasis()` feeds Markdown, JSON-LD and meta descriptions
F4	The `/locations` page family **does not exist** — including the Footer link on every page of the site	FIXED Repointed at the real depot index
F5	140 depot records were really **55 depots**; 96 are virtual. The naive list showed five city names all linking to Atlanta as separate depots	FIXED Deduplicated; virtual entries became the parent's service area
F6	The homepage `SearchAction` pointed at `/products?q=` — a route that has never existed, with a parameter the listing page did not read	FIXED Listing search made URL-addressable, then the target restored and verified
F7	The listing page had **no `<h1>` at all**; depot pages had two identical ones	FIXED Both, the second in the pipeline so one change covers ~1,700 pages
F8	The listing grid and the PDP spec/FAQ tabs are **client-rendered only** — a fetch of the shop returned zero product names	FIXED Server-rendered fallbacks from the same data
F9	`next build` was **already failing** on the branch before this work — a `new Date()` in the Footer	FIXED Confirmed pre-existing by building the stashed baseline
F10	The Merchant feed **cannot be prerendered** — it scans ~10,000 documents, over the framework's build-time cache timeout	FIXED Deferred to runtime. But see F12 — the runtime cost turned out far worse than first measured
F11	The new 404 guard **404'd /llms.txt and /llms-full.txt**. The audit reported it as "all 0 links resolve" — a green tick on a broken file	FIXED Both the bug and the vacuous pass. Caught by the audit script, before release
F12	**The product feeds are not cached at all.** The ~10,000-product array exceeds the 2 MB cache-entry ceiling, so every `'use cache'` write is silently dropped and every request rebuilds from scratch — **3m08s cold, 3m38s warm**, measured on a production build. The same applies to the product sitemap (2.6 MB)	OPEN Needs an architectural decision — see below. This corrects the "~40s, cached hourly" figure reported on 10 Aug

Two things now need a decision, not more code

Both are finished as far as engineering goes. Neither can be closed without someone outside the codebase choosing something.

1 · The product feeds take about three minutes per request DECIDE BEFORE LAUNCH

The catalog is roughly 10,000 products, and the assembled array is 2.6 MB — above the framework's 2 MB ceiling for a single cache entry. The cache write is dropped without an error, so the `'use cache'` directive on these functions is doing nothing and never has. Google Merchant's scheduled fetch will time out long before three minutes.

Three ways out, in rough order of effort: cache a **slimmer projection** (only the fields the feed emits) so the entry fits; **pre-generate the feeds on a schedule** and serve the file from object storage; or install a **custom cache handler** with no 2 MB limit. The middle option also fixes the sitemap, and is the one I would pick — but it changes how the feeds are deployed, so it is your call.

2 · Agent quote requests reach Redis, not the sales team WIRE BEFORE LAUNCH

The endpoint validates, rate-limits, stores and acknowledges — and the requests are visible at `/admin/quote-requests`. But `deliverQuoteRequest()` is a stub: nothing emails, nothing reaches the CRM. Until someone points it at the real pipeline, an agent-submitted request will sit unanswered, which is worse than not accepting one. Tell me which system it should land in and it is a small change.

Blocked — the last 4 tasks, and no amount of engineering closes them

TASK	OWNER	WHAT IS NEEDED
`T3.8`	**Sales**	Real return-policy terms — the return window and conditions. Shopping agents check this field before recommending a listing. A guessed policy in structured data is worse than none, so this stays open
`T2.4`	**Backend team**	A Django-side `agent_summary` field, so the ~1,700 WordPress pages can carry a plain-language summary. The 16 native pages already have one, editable in the admin
`T7.3`	**Content authors**	Alt text on images inside WordPress-authored markup — 10 of 190 on a sampled depot page. Writing it here would mean inventing descriptions of images we cannot see
`T8.5`	**Anyone, 15 min/month**	A citation baseline: ask ChatGPT, Claude, Perplexity and Google AI Overviews six fixed buying questions against the live site, record whether we are cited. The question set and results table are ready in the tracker

Verification

- ✓ `tsc`, `eslint` and `next build` all clean.
- ✓ `npm run audit:agentic` — **45/45 checks**, all 123 llms.txt links verified against soft 404s as well as hard ones, and every machine-readable route asserted reachable.
- ✓ JSON-LD extracted and parsed on every page type; Markdown verified via both suffix and content negotiation.
- ✓ The 404 change tested against 39 real pages before release — none broken.
- ✓ Rate limiting proved under load: 75 parallel requests → exactly 60 served, 15 refused.
- ✓ JSONL feed audited end to end: 10,192 products, zero missing a price basis or freshness stamp.
- ✓ The quote endpoint exercised for all three outcomes — missing fields, bad zip, and a successful submission.